IMAT: Industrial Mobility & Automotive Technology

Master Product Architecture & Strategic Blueprint

---

1. INTRODUCTION

1.1 What is IMAT?

IMAT (Industrial Mobility & Automotive Technology) is a GST-compliant, inventory-first, supplier-driven operating system engineered specifically for the Indian automobile and auto-parts ecosystem. It bridges the gap between chaotic physical supply chains and rigorous financial compliance. IMAT transforms unstructured data (messy supplier invoices, WhatsApp receipts, and fragmented catalogs) into a strictly reconciled, audit-ready database of inventory and ledgers.

1.2 Why IMAT Exists (The Problem Statement)

The Indian auto-parts aftermarket is a multi-billion dollar ecosystem running on fragmented, high-friction processes. Existing ERPs (like generic accounting software) assume a perfect world where catalogs are standardized, pricing is static, and invoices are flawless.

The reality is operational chaos:

- **Supplier Inconsistency:** Suppliers send invoices with varying part numbers, missing HSN codes, and ambiguous descriptions.
- **Dynamic Pricing:** Prices fluctuate daily. An invoice might apply a lump-sum discount across 50 items, making unit-cost calculation a nightmare.
- **Unit Ambiguity:** Products are sold in pieces, boxes, sets, and buckets interchangeably.
- **Regulatory Rigidity:** The government demands strict GST compliance (Input Tax Credit tracking), but the ground reality involves a mix of "Pakka" (White/GST) and "Kaccha" (Grey/Unregistered) billing.

IMAT exists because **inventory is money**. If a distributor cannot accurately land the cost of a spark plug—factoring in apportioned discounts and excluding GST from the inventory valuation—they cannot calculate true profitability. IMAT embraces the chaos at the input layer and enforces absolute strictness at the database layer.

1.3 Who it is Built For

- **Auto-Parts Distributors & Wholesalers:** Managing thousands of SKUs and complex supplier credit terms.

- **Large Retail Dealers:** Needing to track fast-moving vs. dead stock across physical shelves.
- **Multi-Brand Service Centers:** Requiring precise part-to-vehicle mapping and real-time stock availability.

1.4 Core Philosophy

1. **Inventory-First:** Every financial transaction must tie back to a physical stock movement.
2. **Supplier-Driven Ingestion:** The system adapts to how suppliers bill, rather than forcing suppliers to change their behavior.
3. **Audit-Ready by Default:** Immutable ledgers, ACID-compliant database transactions, and strict separation of taxable values from GST amounts.

---

2. CORE SYSTEM ARCHITECTURE

IMAT is built on a modern, decoupled microservices architecture designed for scale, resilience, and data integrity.

- **Frontend (The Command Center):** A React/Vite-based Single Page Application (SPA). It provides a high-performance, dark-mode optimized interface for rapid data entry and dashboard monitoring.
- **Backend API (The Traffic Cop & Business Logic):** A Node.js/Express service that handles authentication, complex transaction routing, tax computation (the Two-Pass Engine), and database communication.
- **OCR & AI Engine (The Chaos Filter):** A dedicated Python (FastAPI) microservice integrating Google Gemini's multimodal AI to read, parse, and structure messy PDF and image-based invoices into JSON payloads.
- **Database (The Vault):** PostgreSQL acts as the single source of truth, utilizing ACID compliance, strict foreign key constraints, and custom CHECK constraints to ensure bad data never corrupts the ledger. MongoDB is utilized in parallel for flexible, unstructured log storage and session management.
- **Storage (The Filing Cabinet):** Cloudflare R2 provides S3-compatible, globally distributed storage for immutable physical receipts and original invoice PDFs.

---

3. FEATURE BREAKDOWN (DETAILED)

3.1 Authentication & User Roles

Purpose: To secure the system against unauthorized access and ensure that destructive actions are heavily gated.

Key Features:

- Role-Based Access Control (RBAC).
- Cryptographic OTP generation with a 60-second cooldown to prevent SMS/Email spamming.
- JWT-based session management with strict HTTP-only cookies.
- Emergency "Kill Switch" to invalidate all active sessions globally.

Workflows:

1. User enters credentials. System validates and checks for account locks (triggered after 5 failed attempts).
2. System generates a secure OTP, hashes it, and dispatches it via the Email Gateway.
3. User submits OTP. System validates the hash, provisions a JWT, and sets a secure browser cookie.

Edge Cases Handled:

- *Timing Attacks:* The system compares hashes using a dummy fallback even if the user doesn't exist, preventing attackers from guessing valid usernames based on response times.
- *Concurrent Logins:* The system tracks active sessionId to allow admins to force-logout an employee's lost device.

Data Model (High Level):

- Users (id, username, password_hash, role, tenant_id, otp_hash, lock_until).
- Sessions (user_id, session_token, ip_address, expires_at).

3.2 Supplier Management

Purpose: To maintain a unified database of vendors, tracking their regulatory compliance (GSTIN) and financial standing (Ledger balances).

Key Features:

- Supplier profiling with state-code mapping (critical for CGST/SGST vs. IGST calculations).
- GSTIN validation and strict uniqueness constraints.
- Supplier-specific payment terms and upfront payment traceability.

Workflows:

1. Admin creates a supplier, entering GSTIN, Address, and Contact.
2. The system determines the supplier's state code and compares it to the host Company's state code to flag them as Intra-state or Inter-state.
3. Accountant records ad-hoc payments to the supplier, uploading a UTR/Cheque receipt to Cloudflare R2.

Edge Cases Handled:

- *Unregistered Suppliers:* System allows creation of suppliers without GSTINs but permanently flags them. The Purchase Engine will later forcefully block GST entry for these suppliers.
- *Duplicate Entities:* PostgreSQL unique constraints block the creation of multiple suppliers with the same GSTIN, preventing fragmented ledgers.

Data Model (High Level):

- Suppliers (id, supplier_name, gstin_uin, state_code, current_balance).
- Supplier_Payments (id, supplier_id, amount, payment_mode, receipt_url, created_at).

3.3 Product Management (VERY DEEP)

Purpose: To resolve the chaos of auto-parts nomenclature by maintaining a clean, deduplicated Master Catalog.

Key Features:

- Universal SKU generation (System-controlled).
- Manufacturer Part No. tracking.
- Category and Sub-category mapping.
- Dynamic unit handling (PCS, BOX, SET).

Workflows:

1. During an invoice upload, the OCR detects a part: WD-40 100ml.
2. The operator searches the Master Catalog. If it exists, they map the OCR text to the existing SKU-00012.
3. If new, the system generates a new SKU, requests an HSN/SAC code, and adds it to the Master Catalog permanently.

Edge Cases Handled:

- *Supplier Nomenclature Variance:* Supplier A calls it BOSCH SPARK PLG. Supplier B calls it PLUG-SPRK-BSCH. IMAT maps both raw text inputs to the same internal SKU-00451.
- *Unit Conversions:* The system standardizes the base inventory unit (e.g., PCS) even if a supplier bills by the BOX.

Data Model (High Level):

- Categories (id, name, parent_id).
- Products (product_id, sku_code, part_no, description, hsn_sac, base_unit, category_id).

3.4 Purchase / Inward System

Purpose: The most complex and critical module in IMAT. It translates messy physical invoices into strictly reconciled inventory and financial records.

Key Features:

- Python-driven AI OCR for automated data extraction.
- Staged (Draft) environment for human-in-the-loop verification.
- Two-Pass Tax Engine for mathematically perfect ledger entries.
- "Kaccha" (Grey) vs. "Pakka" (GST) dynamic switching.

Workflows:

1. **Ingestion:** Operator uploads a PDF invoice. Python AI extracts line items, totals, and supplier details.
2. **Staging:** Data is held in a staged_purchases table. Operator reviews the digitized data against the physical image (side-by-side UI).
3. **Mapping:** Operator maps extracted text to Master SKUs.
4. **Commitment:** The system executes an ACID-compliant transaction: updating the supplier ledger, creating purchase line items, writing stock movements, and logging the action.

The Two-Pass Tax Engine (Mathematical Architecture):

Because auto-parts suppliers often give lump-sum discounts at the bottom of an invoice, determining the actual landing cost of a single bolt is mathematically complex.

- **Pass 1 (Gross Calculation):** Compute Gross = Qty * UnitPrice for every item. Sum these to get Total_Gross.
- **Pass 2 (Apportionment):** For every item, calculate its weight: $Weight = \frac{ItemGross}{TotalGross}$.
- **Pass 3 (Net Calculation):** Calculate Net Taxable = Gross - (Total_Overall_Discount * Weight).
- **Pass 4 (Tax Application):** Apply GST *only* to the Net Taxable amount.
- *Result:* Perfect matching with the physical invoice to the exact decimal, ensuring Input Tax Credit is claimed accurately.

Edge Cases Handled:

- *The "Double Envelope" Bug:* If a user rapidly clicks submit, or frontend state duplicates array inputs (e.g., ["neft", "neft"]), the backend sanitizes the inputs to prevent database constraint violations.
- *GST Conflicts:* If a user attempts to log GST percentages for an unregistered supplier, the system hard-blocks the transaction.
- *Missing Invoice Numbers:* If a registered supplier's invoice number is missing, the system forces the user to declare it as a "Grey" (Kaccha) purchase, instantly zeroing out all GST

implications to protect the company's compliance record.

Data Model (High Level):

- Staged_Purchases (id, ocr_raw_data, status, file_path).
- Purchases (id, supplier_id, invoice_number, purchase_type, total_taxable, total_gst, grand_total).
- Purchase_Items (id, purchase_id, product_id, quantity, unit_price, apportioned_discount, gst_amount).

3.5 Inventory System (CORE)

Purpose: To maintain an absolute, auditable record of every physical item on the shelves.

Key Features:

- Strict FIFO (First-In-First-Out) stock valuation.
- Immutable stock_movements ledger (no "editing" stock, only "adjusting" via opposite entries).
- Separation of Inventory Valuation from Tax (GST is excluded from stock value).

Workflows:

1. Purchase is committed (see 3.4). System writes +X to stock_movements linked to the purchase_id.
2. Sales occurs. System writes -Y to stock_movements linked to the sale_id.
3. Physical audit reveals a missing spark plug. Manager executes a "Stock Adjustment", writing -1 to stock_movements with the reason code SHRINKAGE.

Edge Cases Handled:

- *Negative Stock Prevention:* Database constraints prevent outbound stock movements that would push total quantity below zero, forcing users to inward missing purchases first.

Data Model (High Level):

- Stock_Movements (id, product_id, source_type, source_id, quantity_change, unit_cost).
- *(Note: Current stock is always dynamically calculated as SUM(quantity_change), ensuring mathematical purity).*

3.6 Sales / Customer System

Purpose: To facilitate outbound inventory movement and manage buyer ledgers.

Key Features:

- Customer profiling (B2B wholesale vs. B2C retail).
- Credit limit enforcement.
- Automated Outward Invoicing.

Workflows:

1. Operator selects a customer and scans/selects SKUs.
2. System validates available stock in real-time.
3. System calculates Outward GST (CGST/SGST vs IGST based on customer state code).
4. Invoice is generated, stock is deducted, and customer ledger is debited.

Edge Cases Handled:

- *Over-Credit:* If a sale pushes a B2B customer past their defined credit limit, the system requires Manager override or upfront partial payment.

Data Model (High Level):

- Customers (id, name, gstin, credit_limit, balance).
- Sales (id, customer_id, invoice_no, total_amount).

3.7 Ledger & Accounting (GST-aware)

Purpose: To maintain a unified chart of accounts that strictly segregates operational money from government money (Tax).

Key Features:

- Supplier Accounts Payable (AP) tracking.
- Customer Accounts Receivable (AR) tracking.
- FIFO Payment Allocation (mapping a specific ₹10,000 NEFT payment to Invoice A and Invoice B).

Workflows:

1. Supplier balance sits at ₹50,000.
2. Company makes a ₹20,000 NEFT payment.
3. System credits the Supplier Ledger by ₹20,000 and automatically allocates this amount against the oldest unpaid invoices in the database.

Edge Cases Handled:

- *Overpayments:* If an advance payment is made, it sits as an unallocated credit on the supplier's ledger, ready to be automatically consumed by the next inward purchase.

Data Model (High Level):

- Supplier_Payment_Allocations (id, payment_id, purchase_id, allocated_amount).

3.8 Reports & Insights

Purpose: To transform ledger data into actionable business intelligence.

Key Features:

- Real-time Stock Valuation (Quantity $\times$ Moving Average Unit Cost).
- Fast/Slow moving SKU identification.
- Supplier profitability analysis.

3.9 Audit & Traceability

Purpose: Absolute accountability. When data looks wrong, the system can prove exactly who touched it, when, and from what IP address.

Key Features:

- MongoDB-backed unstructured Audit Trails.
- Tracking of old values vs. new values on edits.

---

4. IMAT V1 (CURRENT CAPABILITIES)

4.1 What is Already Built (The Foundation)

IMAT V1 is a highly stable, robust core system. It successfully executes the hardest part of the auto-parts business: **Input Ingestion.**

- The Python-to-Node bridge for OCR extraction is live and operational.
- The Staged Purchase verification UI (Side-by-side document comparison) works flawlessly.
- The Two-Pass Tax Engine dynamically allocates overall discounts to individual SKUs with exact decimal precision.
- Strict PostgreSQL constraints protect the ledger from bad data (e.g., blocking array inputs or duplicate GSTINs).
- R2 Cloud Storage safely archives original documents.

4.2 What is Intentionally Simplified

- **Customer ledgers:** Currently function as basic T-accounts without deep FIFO allocation of payments.
- **Pricing Engine:** Currently relies on moving average cost; it does not yet utilize complex margin-based auto-pricing for outbound sales.

4.3 Known Limitations

- The OCR engine requires human verification. If a supplier sends a completely illegible handwritten note, the OCR will fail gracefully, requiring manual fallback entry.
- Demand forecasting is manual; the system reports what is low, but does not autonomously generate Purchase Orders.

---

5. IMAT V2 (FUTURE ROADMAP)

IMAT V2 transforms the platform from a "System of Record" into a "System of Intelligence."

5.1 V2 Core Upgrades

- **Automated SKU Matching (AI Confidence):** The Python engine will learn from historical mappings. If "WD40 (32)" is mapped to SKU-001 three times, the 4th time, the system will bypass human verification and auto-map it with 99% confidence.
- **Multi-Warehouse Architecture:** Tracking inventory across different physical locations, including in-transit stock.

5.2 V2 Advanced Features

- **Dynamic Pricing Intelligence Engine:** Automatically adjust B2B sales prices based on real-time landed costs, desired margin matrices, and competitor APIs.
- **Predictive Ordering:** Analyze seasonal trends (e.g., wiper blades in monsoon) and automatically draft Purchase Orders to preferred suppliers.

5.3 V2 Strategic Direction (The Moat)

- **The B2B Network Effect:** As more distributors use IMAT, the system's global Master Catalog becomes the definitive auto-parts database in India.
- **Marketplace Layer:** Allowing an IMAT user (Retailer) to view live inventory of another IMAT user (Distributor) and execute seamless intra-system B2B orders without manual invoicing.

---

6. DIFFERENTIATION

IMAT is built on the premise that traditional software fails in the auto-parts industry.

Why IMAT is NOT a Generic Accounting Tool (like Tally/Marg):

Generic tools are accounting-first. They treat inventory as a text field attached to a financial transaction. IMAT is **inventory-first**. You cannot make a ledger entry without the system knowing exactly which physical item moved, where it came from, and how much discount was apportioned to it.

Why IMAT is NOT a Generic ERP (like Zoho/Odoo):

Generic ERPs require pristine input data. They force the business to adapt to the software. IMAT adapts to the business. It anticipates that a supplier will send a dirty WhatsApp image of a Kaccha bill, and it provides the exact tooling (OCR, Staging, Grey-bill flagging) to ingest it safely without corrupting the compliance ledger.

---

7. IDEAL USER JOURNEY

- **Day 1 (Foundation):** The business owner logs in, updates the Company Profile (State Code, GSTIN), and creates User Roles for the accountant and store operator.
- **Day 2 (Master Mapping):** High-volume suppliers are created. The existing Excel-based product catalog is imported into the Master Products table.
- **Day 3 (The First Ingestion):** The operator uploads 10 backlogged supplier invoices via the OCR module. They spend the day in the "Staged Purchases" screen, mapping supplier jargon to the clean Master SKUs.
- **Day 7 (Inventory Stabilization):** Physical shelf stock begins to match the system stock perfectly. The business owner can instantly see the exact landed cost of any part, factoring in the complex discounts applied 4 days ago.
- **Day 30 (Financial Clarity):** The accountant pulls the GSTR report. It perfectly matches the Input Tax Credit claimed because IMAT forcefully prevented GST from being applied to unregistered supplier invoices. The ledger is clean.

---

8. CONCLUSION

IMAT is not just a software product; it is a category creator for the Indian automotive aftermarket. By accepting the inherent chaos of the supplier ecosystem and building rigid, intelligent filters to process that chaos, IMAT provides unparalleled operational clarity.

For the **distributor**, it stops profit leakage caused by miscalculated landing costs.

For the **accountant**, it guarantees compliance in a highly scrutinized tax environment.

For the **investor**, it represents a highly scalable, sticky B2B platform that builds a massive data moat with every invoice processed.

IMAT V1 has laid the unbreakable foundation. IMAT V2 will build the intelligent, interconnected future of auto-parts commerce.